

Multimodal Autonomous Agent Application

Multimodal Autonomous Agent Application

1. Concept Introduction
 - 1.1 What is an "Autonomous Agent"?
 - 1.2 Implementation Principle Summary
2. Code Analysis
 - Key Code
 1. Agent Core Workflow (`largetmodel/utis/ai_agent.py`)
 2. Task Planning and LLM Interaction (`largetmodel/utis/ai_agent.py`)
 - Code Analysis
3. Practical Operations
 - 3.1 Configure Online LLM
 - 3.2 Start and Test the Feature

1. Concept Introduction

1.1 What is an "Autonomous Agent"?

In the `largetmodel` project, the **Multimodal Autonomous Agent** is the most advanced form of intelligence. It no longer simply responds to a single instruction from the user, but can **autonomously think, plan, and consecutively call multiple tools to accomplish a complex goal**.

The core of this feature is the `agent_call` tool or the **ToolChainManager** behind it. When a user makes a complex request that cannot be completed with a single tool call, the autonomous agent starts up.

1.2 Implementation Principle Summary

The autonomous agent implementation in `largetmodel` follows the industry-standard **ReAct (Reason + Act)** paradigm. Its core idea is to emulate the human problem-solving process, cycling between "thinking" and "action".

1. **Reason:** When the agent receives a complex goal, it first calls a powerful language model (LLM) to "think". It asks itself: "To accomplish this goal, what should my first step be? Which tool should I use?" The LLM's output is not the final answer, but an action plan.
2. **Act:** The agent executes the corresponding action based on the LLM's reasoning result—calling `ToolsManager` to run the specified tool (e.g., `visual_positioning`).
3. **Observe:** The agent obtains the result of the previous action ("observation"), for example, `{"result": "Found the cup, position at [120, 300, 180, 360]"}`.
4. **Think Again:** The agent submits the observation result, together with the original goal, to the LLM for the second round of "thinking". It asks itself: "I've found the cup's position. Next, to know its color, what should I do?" The LLM might generate a new action plan, such as `{"thought": "I need to analyze the image area where the cup is located to determine the color", "action": "seewhat", "args": {"crop_area": [120, 300, 180, 360]}}`.

This **Think -> Act -> Observe** loop continues until the original goal is achieved. Only then does the agent generate and output the final answer.

2. Code Analysis

Key Code

1. Agent Core Workflow (`largemodel/utils/ai_agent.py`)

The `_execute_agent_workflow` function is the main execution loop of the Agent, defining the core flow of "Plan -> Execute".

```
# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...

    def _execute_agent_workflow(self, task_description: str) -> Dict[str, Any]:
        """
        Executes the agent workflow: Plan -> Execute -> Reflect -> Adjust.
        """
        try:
            # Step 1: Task Planning.
            self.node.get_logger().info("AI Agent starting task planning phase")
            plan_result = self._plan_task(task_description)
            self.node.get_logger().info(f"Task planning result: {plan_result}")

            if not plan_result["success"]:
                self.node.get_logger().error(f"Task planning failed: {plan_result.get('message', 'Unknown error')}")
                return plan_result

            self.task_steps = plan_result["steps"]
            self.node.get_logger().info(f"Task planning completed with {len(self.task_steps)} steps")

            # Send detailed planning results to the frontend.
            if self.feedback_callback:
                plan_details = "\n".join([f" - Step {s['id']}: {s['description']}" for s in self.task_steps])
                self.feedback_callback(f"🛠️ AI Agent task planning complete, {len(self.task_steps)} steps total:\n{plan_details}")

            # Step 2: Execute Task Steps.
            execution_results = []
            step_outputs = {} # Store step outputs for subsequent steps.
            tool_outputs = [] # Store outputs in ToolOutput format for toolchain data passing.

            for i, step in enumerate(self.task_steps):
                step_info = f"▶️ Execute Step {i+1}/{len(self.task_steps)}: {step['description']}"
                self.node.get_logger().info(step_info)
                if self.feedback_callback:
                    self.feedback_callback(step_info)

                # Skip steps without a tool.
```

```

        if not step.get("tool") or step.get("tool").strip() == "":
            self.node.get_logger().warn(f"Step {i+1} has no tool
specified, skipping execution")
            execution_results.append({
                "success": True,
                "step_id": step.get("id"),
                "description": step["description"],
                "message": "skipped (no tool)"
            })
            continue

        # Step 1: Process parameters before execution.
        # This method finds {{...}} references and replaces them with
        actual outputs from previous steps.
        try:
            processed_parameters =
self._process_step_parameters(step.get("parameters", {}), tool_outputs)
            step["parameters"] = processed_parameters # Update
parameters in the step.
        except Exception as e:
            self.node.get_logger().error(f"Parameter processing failed,
step {i+1} will be skipped: {e}")
            step_result = {
                "success": False,
                "step_id": step.get("id"),
                "description": step["description"],
                "error": f"Parameter processing failed: {e}",
                "message": f"Step execution failed: {e}"
            }
            execution_results.append(step_result)
            # If parameter processing fails, terminate the entire task.
            return {"success": False, "message": f"Task terminated due
to parameter processing failure in step {i+1} ('{step['description']}'): {e}",
"results": execution_results}

        # Record the step in the current task.
        step_record = {
            "step_id": step.get("id"),
            "description": step["description"],
            "start_time": datetime.now()
        }

        # Step 2: Execute the step with processed parameters.
        step_result = self._execute_step(step, tool_outputs)
        execution_results.append(step_result)

        # Step 3: Save the output for subsequent steps to reference.
        if step_result.get("success"):
            # Use the complete ToolOutput object returned by the tool
            adapter.

            if step_result.get("tool_output"):
                tool_outputs.append(step_result["tool_output"])
                self.node.get_logger().info(f"Step {i+1} output added to
the tool chain passing list")
            else:
                # Also add failed steps to the tool output list, but mark
                them as failed.
                if step_result.get("tool_output"):

```

```

        tool_outputs.append(step_result["tool_output"])
    else:
        from utils.tool_chain_manager import ToolOutput
        tool_output = ToolOutput(
            tool_name=step.get("tool"),
            success=False,
            data=None,
            metadata={"step_id": step.get("id"), "description":
step.get("description")},
            error_message=step_result.get("error", "Step
execution failed and did not generate a standard output object")
        )
        tool_outputs.append(tool_output)

    # Complete the step record.
    step_record["end_time"] = datetime.now()
    step_record["success"] = step_result.get("success")
    step_record["result"] = step_result
    if self.current_task:
        self.current_task["steps"].append(step_record)

    # If a step fails, abort the entire task.
    if not step_result.get("success", False):
        self.node.get_logger().error(f"Step {i+1} execution failed:
{step_result.get('message', 'Unknown error')}")
        self.node.get_logger().error(f"Task '{task_description}'
terminated early due to a failed step.")
        # Return a final result with failure information.
        return {
            "success": False,
            "message": f"Task terminated because step
'{step['description']}' failed.",
            "steps_executed": i + 1,
            "results": execution_results
        }
    else:
        self.node.get_logger().info(f"Step {i+1} executed
successfully")

        # Send successful tool execution result.
        if self.feedback_callback:
            tool_output = step_result.get("tool_output")
            if tool_output:
                feedback =
self._format_tool_output_for_feedback(tool_output)
                self.feedback_callback(feedback)

        summary = self._summarize_execution(task_description,
execution_results)

    # Check for generated documents.
    generated_files = []
    for result in execution_results:
        if result.get("success") and "Saved to:" in
str(result.get("result", "")):
            # Extract file path.
            result_text = str(result.get("result", ""))
            if "Saved to:" in result_text:
                file_path = result_text.split("Saved to:")[1].strip()

```

```

        generated_files.append(file_path)

    return {
        "success": True,
        "message": summary,
        "steps_executed": len(execution_results),
        "results": execution_results,
        "generated_files": generated_files
    }

except Exception as e:
    self.node.get_logger().error(f"AI Agent workflow execution
exception: {e}")
    import traceback
    self.node.get_logger().error(f"Exception stack:
{traceback.format_exc()}")
    return {"success": False, "message": f"workflow execution failed: {

```

2. Task Planning and LLM Interaction (`largemodel/utils/ai_agent.py`)

The `_plan_task` function's core is constructing a precise Prompt, utilizing the large model's own reasoning capability to generate a structured execution plan.

```

# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...
    def _plan_task(self, task_description: str) -> Dict[str, Any]:
        """
        Uses the large model for task planning and decomposition.
        """
        try:
            self.node.get_logger().info("Starting to construct planning prompt")

            # Dynamically generate tool list and their descriptions
            tool_descriptions = []
            for name, adapter in
self.tools_manager.tool_chain_manager.tools.items():
                # Get high-quality descriptions directly from schema
                description = adapter.input_schema.get("description", "No
description available.")

                # Extract parameter names for hints
                props = adapter.input_schema.get("properties", {})
                params = ", ".join(props.keys())

                tool_descriptions.append(f"- {name}({params}): {description}")

            available_tools_str = "\n".join(tool_descriptions)

            planning_prompt = f"""
As a professional task planning Agent, please decompose the user task into a
series of specific, executable JSON steps.

*** Available Tools:***
{available_tools_str}

```

```

## Core Rules:
1. Data Passing: When subsequent steps need to use the output from previous steps, must use the `{{{steps.N.outputs.KEY}}}` format for referencing.
    - `N` is the step ID (starting from 1).
    - `KEY` is the specific field name in the output data of the previous step.
    - After `outputs`, it can be `data` (main data) or `metadata.sub_key` (metadata).
2. JSON Format: Must strictly return a JSON object, do not include any markdown wrapping (such as ```json```).
3. Tool Selection: Strictly select the most suitable tool based on tool descriptions.

## Output Format:
```json
{
 "steps": [
 {
 "id": 1,
 "description": "Brief Chinese description of the first step",
 "tool": "Tool name",
 "parameters": { "param1": "value1" }
 },
 {
 "id": 2,
 "description": "Brief Chinese description of the second step",
 "tool": "Another tool",
 "parameters": { "paramA": "{{{steps.1.outputs.data}}}", "paramB": "{{{steps.1.outputs.metadata.file_path}}}" }
 }
]
}

```

### # Example: "First look around, then write what you see into a document"

```

{
 "steps": [
 {
 "id": 1,
 "description": "First, use the seewhat tool to observe the current environment.",
 "tool": "seewhat",
 "parameters": { }
 },
 {
 "id": 2,
 "description": "Then, write the observed environment description into a TXT document.",
 "tool": "write_document",
 "parameters": {
 "format": "txt",
 "title": "Environment Observation Report",
 "content": "{{{steps.1.outputs.data}}}",
 "filename": "env_report.txt"
 }
 }
]
}

```

```
]
}}
```

### # User Task:

```
{task_description}
"""
```

```
Call the large model for planning
self.node.get_logger().info("Start calling LLM for task planning")
messages_to_use = [{"role": "user", "content": planning_prompt}]
result = self.node.model_client.infer_with_text("",
message=messages_to_use)

if isinstance(result, dict):
 response_text = result.get('response', '')
else:
 response_text = str(result)

Process markdown-formatted JSON response
json_text = response_text
if "```json" in response_text:
 # Extract JSON part from markdown
 start_marker = "```json"
 end_marker = "```"
 start_idx = response_text.find(start_marker)
 if start_idx != -1:
 start_idx += len(start_marker)
 end_idx = response_text.find(end_marker, start_idx)
 if end_idx != -1:
 json_text = response_text[start_idx:end_idx].strip()
 self.node.get_logger().info(f"Extracted JSON text:
{json_text}")

Parse JSON response
try:
 plan_data = json.loads(json_text)
 steps = plan_data.get("steps", [])

 self.node.get_logger().info(f"Parsed steps: {steps}")

 if not steps:
 self.node.get_logger().error("Task planning failed: no valid
steps were generated")
 return {"success": False, "message": "Task planning failed: No
valid steps generated"}

 return {"success": True, "steps": steps}

except json.JSONDecodeError as e:
 # If JSON parsing fails, try to extract step information
 self.node.get_logger().error(f"JSON parsing failed: {e}, response
content: {response_text}")
 return {"success": False, "message": f"Task planning failed: Response
format error - {str(e)}"}

except Exception as e:
```

```
self.node.get_logger().error(f"Task planning exception: {e}")
return {"success": False, "message": f"Task planning failed: {str(e)}"}
```

### #### 3. Parameter Processing and Data Flow Implementation (`largemodel/utils/ai\_agent.py`)

The `\_process\_step\_parameters` function is responsible for parsing placeholders and implementing data flow between steps.

```
```python
# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...
    def _process_step_parameters(self, parameters: Dict[str, Any],
previous_outputs: List[Any]) -> Dict[str, Any]:
    """
        Parses parameter dictionary, finds and replaces all {{...}} references
        using regex.
        This implementation can handle multiple references within a single
        string.
    """
    processed_params = parameters.copy()
    pattern = re.compile(r"\{\{steps\.(\d+)\.outputs\.(.+?)\}\}")

    for key, value in processed_params.items():
        if not isinstance(value, str):
            continue

        # Use an inner function as the replacement argument for re.sub to
        execute find logic for each match.
        def replacer(match):
            step_id = int(match.group(1))
            path = match.group(2).strip() # .strip() ensures the path has no
            extra spaces.

            self.node.get_logger().info(f"Found reference in parameter
            '{key}': step_id={step_id}, path='{path}'")

            if not (1 <= step_id <= len(previous_outputs)):
                raise ValueError(f"Invalid step ID referenced: {step_id}
            (available steps: {len(previous_outputs)})")

            # Get the corresponding step output.
            target_output = previous_outputs[step_id - 1]

            # Extract the real value from the ToolOutput object based on the
            path.
            retrieved_value = self._get_value_from_path(target_output, path)

            if retrieved_value is None:
                available_keys = list(target_output.to_dict().keys())
                available_metadata_keys =
            list(target_output.to_dict().get('metadata', {}).keys())
```

```

        raise ValueError(f"Path '{path}' not found in the output of
step {step_id}. Available top-level keys: {available_keys}. Available metadata
keys: {available_metadata_keys}")

        # The returned value must be a string to be embedded in the
original parameter string.
        return str(retrieved_value)

    try:
        # Use re.sub and our replacer function to replace all found
placeholders.

        # Check if the placeholder pattern exists in the original value.
        if pattern.search(value):
            new_value = pattern.sub(replacer, value)

            single_match = pattern.fullmatch(value)
            if single_match:
                step_id = int(single_match.group(1))
                path = single_match.group(2).strip()
                retrieved_value =
self._get_value_from_path(previous_outputs[step_id - 1], path)
                if retrieved_value is not None:
                    processed_params[key] = retrieved_value
                    self.node.get_logger().info(f"Parameter '{key}' has
been directly replaced with the value (type: {type(retrieved_value).__name__})")
                else:
                    raise ValueError(f"Path '{path}' not found in the
output of step {step_id}")
            else:
                processed_params[key] = new_value
                self.node.get_logger().info(f"Parameter '{key}' updated.
Old value: '{value}', New value: '{new_value}'")

    except ValueError as e:
        raise e

    return processed_params

```

Code Analysis

The AI Agent is the "central brain" of the system, transforming high-level, sometimes vague user tasks into a series of precise, ordered tool calls. Its implementation does not depend on any specific model platform but is built on a universal, extensible architecture.

1. **Dynamic Task Planning:** The core capability of the Agent lies in the `_plan_task` function. It does not rely on hardcoded logic but dynamically generates task plans by interacting with the large model.
 - **Self-awareness and Prompt Construction:** At the beginning of planning, the Agent first checks all currently available tools and their descriptions. Then, it packages this tool information, the user task, and strict rules (such as the data passing format) into a highly structured `planning_prompt`.
 - **Model as Planner:** This Prompt is sent to a general-purpose text large model. The model reasons based on the provided context and returns a JSON-formatted action plan containing multiple steps. This design is highly extensible: when a new tool is

added or an existing tool is modified in the system, the Agent's planning capability automatically updates without modifying the Agent's own code.

2. **Tool Chain and Data Flow:** Real-world tasks often require the collaboration of multiple tools. For example, "take a photo and describe it" requires the output of the "take photo" tool (image path) to be used as the input for the "describe" tool. The AI Agent elegantly implements this through the `_process_step_parameters` function.
 - **Data Reference Placeholders:** During the planning phase, the large model embeds special placeholders, such as `{{steps.1.outputs.data}}`, in parameter values where data needs to be passed.
 - **Real-time Parameter Replacement:** In the main loop of `_execute_agent_workflow`, before executing each step, `_process_step_parameters` is called. It uses regular expressions to scan all parameters of the current step. Once a placeholder is found, it finds the corresponding data from the output list of previous steps and performs real-time replacement. This mechanism is the key to achieving complex task automation.
3. **Supervised Execution and Fault Tolerance:** `_execute_agent_workflow` forms the main execution loop of the Agent. It strictly follows the planned step order, executes each action in sequence, and ensures correct data passing between steps.
 - **Atomic Steps:** Each step is treated as an independent "atomic operation". If any step fails to execute, the entire task chain immediately stops and reports an error. This ensures the system's stability and predictability, avoiding continued execution in an erroneous state.

To summarize, the AI Agent's universal implementation demonstrates an advanced software architecture: it doesn't solve problems directly, but builds a framework that allows an external, general-purpose reasoning engine (large model) to solve problems. Through the two core mechanisms of "Dynamic Planning" and "Data Flow Management", the Agent can orchestrate a series of independent tools into complex workflows capable of accomplishing high-level tasks.

3. Practical Operations

3.1 Configure Online LLM

1. **Update the API key in the configuration file, open the model interface configuration file** `large_model_interface.yaml`:

```
vim ~/yahboomcar_ws/src/largemodel/config/large_model_interface.yaml
```

2. **Enter your API Key:**

Find the corresponding section and paste your copied API Key here. Here's an example using Tongyi Qianwen configuration:

```
# large_model_interface.yaml

## Tongyi Qianwen
qianwen_api_key: "sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx" # Paste your Key here
qianwen_model: "qwen-vl-max-latest" # Can choose model as needed, such as
qwen-turbo, qwen-plus
```

3. **Open the main configuration file** `yahboom.yaml`:

```
vim ~/yahboomcar_ws/src/largemodel/config/yahboom.yaml
```

4. Select the online platform to use:

Modify the `llm_platform` parameter to the platform name you want to use:

```
# yahboom.yaml

model_service:
  ros__parameters:
    # ...
    llm_platform: 'tongyi' #Optional platforms: 'tongyi', 'spark',
    'qianfan', 'openrouter'
```

After modifying the configuration file, you need to recompile in the workspace and source:

```
cd ~/yahboomcar_ws
colcon build && source install/setup.bash
```

3.2 Start and Test the Feature

1. Start the `largetmodel` main program:

Open a terminal and run the following command:

```
ros2 launch largetmodel largetmodel_control.launch.py text_chat_mode:=true
```

2. Send text instructions:

 Due to the official DiGua (RDK) system, Chinese input method can only be used when the system language is set to Chinese. The factory image does not switch languages. You can use Chinese pinyin input via the SSH remote software terminal.

Open another terminal and run the following command:

```
ros2 run text_chat text_chat
```

Then start entering text: "根据当前的环境，生成一张类似场景的图片" (Based on the current environment, generate a similar scene image).

If you want a photo of the real-time situation, you can ask it to take a photo and then generate a similar image based on that photo.

3. Observe the results:

In the first terminal running the main program, you will see log output showing the system received the text instruction, called the `aiagent` tool, and then provided the prompt to the LLM. The LLM will analyze the detailed tool calling steps. For example, with this query, it will call the `seewhat` tool to capture the screen, then provide the captured image to the LLM for parsing. The parsed text will then be fed back to the LLM as the content for the new image generation.